

AI AGENTS

Product Integration & Developer Guide

Version 1.0 · 2025

1. Introduction to AI Agents

An AI Agent is an autonomous software system that perceives its environment, makes decisions, and takes actions to achieve specific goals — often without requiring step-by-step human instructions. Unlike traditional automation, AI agents can adapt to context, learn from feedback, and coordinate multi-step tasks across multiple tools and services.

1.1 What Makes an Agent Different?

Traditional bots follow hard-coded rules. AI Agents combine large language models (LLMs), memory systems, and tool use to reason dynamically. Key differentiators:

- Reasoning — the agent plans before acting, breaking complex tasks into subtasks.
- Memory — agents retain short-term conversation context and long-term knowledge (vector stores, databases).
- Tool Use — agents can call APIs, run code, browse the web, query databases, or invoke other agents.
- Autonomy — given a high-level goal, the agent decides how to achieve it with minimal human intervention.

1.2 Core Agent Architecture

Every AI Agent is built on four foundational layers:

Layer	Component	Role	Example
Perception	Input Handler	Receives user input / events	Chat message, API event
Reasoning	LLM Core	Plans, decides next action	GPT-4, Claude, Gemini

Memory	Context Store	Tracks state and knowledge	Vector DB, Redis
Action	Tool Executor	Calls APIs, runs tasks	Search, Code, Email

2. Types of AI Agents

2.1 Reactive Agents

Reactive agents respond to immediate stimuli without maintaining internal state. They are fast and predictable, ideal for real-time applications.

- Use case: FAQ bots, live customer support triage
- Limitation: Cannot handle multi-step workflows or remember prior context

2.2 Deliberative Agents

Deliberative (or planning) agents reason about goals, form plans, and execute them step by step. They use a Think → Plan → Act loop and can handle complex, long-horizon tasks.

- Use case: Research agents, code generation agents, autonomous data pipelines
- Limitation: Higher latency; requires robust error recovery

2.3 Multi-Agent Systems (MAS)

A network of specialized agents that collaborate to solve problems beyond any single agent's capability. Agents may play roles such as Orchestrator, Executor, Critic, or Specialist.

- Use case: Enterprise workflows (HR onboarding, financial audits, product launches)
- Limitation: Coordination complexity; requires message-passing protocol

2.4 Agentic RAG (Retrieval-Augmented Generation)

Combines retrieval systems (vector databases, search engines) with generative AI so the agent can answer questions grounded in real, up-to-date data rather than purely parametric knowledge.

- Use case: Legal document Q&A, medical knowledge bases, internal wiki assistants

3. Integrating AI Agents into Your Products

3.1 Choosing the Right Agent Pattern

Before building, map your use case to the correct agent pattern:

Product Need	Recommended Pattern	Key Tools Required
--------------	---------------------	--------------------

Answer customer questions	Reactive Agent + RAG	Vector DB, Chat UI
Automate multi-step workflows	Deliberative / Planning Agent	Task queue, APIs
Handle diverse departments	Multi-Agent System	Orchestrator, Brokers
Summarize / analyze documents	Agentic RAG	PDF parser, Embeddings
Write & execute code	Code Execution Agent	Sandbox, Interpreter

3.2 System Prompt Design

The system prompt is the backbone of agent behavior. A well-crafted system prompt defines role, capabilities, constraints, and output format.

Template: Customer Support Agent

```
SYSTEM PROMPT:
You are Aria, a friendly support agent for Acme Corp.
Your role: Help customers resolve billing and shipping issues.
Tools available: lookup_order(order_id), process_refund(order_id, amount)

Rules:
- Always confirm the customer's identity before accessing order data.
- Never issue refunds above $500 without manager approval.
- If unsure, escalate to human with message: "ESCALATE: <reason>"

Output format: Always end responses with next-steps or an ETA.
```

3.3 Tool Registration

Agents gain capabilities through tools. Each tool must be registered with a clear name, description, and typed parameters. Below is an example using the Anthropic API tool-use schema:

```
tools = [
  {
    "name": "lookup_order",
    "description": "Retrieves order status, items, and shipping info.",
    "input_schema": {
      "type": "object",
      "properties": {
        "order_id": {
          "type": "string",
          "description": "The unique order identifier (e.g., ORD-12345)"
        }
      },
      "required": ["order_id"]
    }
  }
]
```

3.4 Memory Management

Agents need memory to be contextually aware across turns and sessions:

Memory Type	Description	Implementation
In-Context	Chat history in LLM context window	messages[] array
External Short-Term	Recent turns stored externally	Redis / Session DB
Long-Term Semantic	Persistent facts & knowledge	Pinecone / Weaviate
Episodic	History of past agent runs	Postgres / MongoDB

3.5 Agentic Loop Implementation

The core agentic loop: the agent receives input, calls tools if needed, processes results, and decides whether to act again or return a final response.

```
async function runAgent(userMessage, conversationHistory) {
  const messages = [...conversationHistory,
    { role: "user", content: userMessage }];

  while (true) {
    const response = await callLLM({ messages, tools });

    if (response.stop_reason === "end_turn") {
      return response.content; // Final answer
    }

    if (response.stop_reason === "tool_use") {
      const toolResult = await executeTool(response.tool_name,
        response.tool_input);
      messages.push({ role: "assistant", content: response.content });
      messages.push({ role: "user", content: toolResult });
    }
  }
}
```

4. Sample Agent Use Cases by Product Type

4.1 E-Commerce: Order Management Agent

Goal: Allow customers to track, modify, and get refunds for their orders via a conversational interface.

- Tools: lookup_order, cancel_order, process_refund, send_email
- Memory: Customer ID stored in session; past orders retrieved from vector store
- Integration: Embed in website chat widget or WhatsApp Business API
- Escalation: Route to human agent if sentiment is negative or issue is unresolved after 3 turns

4.2 Healthcare: Clinical Documentation Agent

Goal: Transcribe doctor-patient conversations and auto-populate structured medical notes in the EHR system.

- Tools: `transcribe_audio`, `extract_entities`, `update_ehr`, `fetch_patient_record`
- Memory: Patient history retrieved from EHR; session context cleared after encounter
- Integration: FHIR API, ambient listening microphone, EHR portal
- Compliance: Outputs reviewed by clinician before commit; audit logs maintained

4.3 Developer Tools: Code Review Agent

Goal: Automatically review pull requests, suggest improvements, flag security issues, and enforce code style.

- Tools: `read_diff`, `run_linter`, `search_codebase`, `post_comment`
- Memory: Project coding standards stored in long-term vector store
- Integration: GitHub / GitLab webhook triggers agent on PR creation
- Output: Inline comments on the PR with severity labels: Info / Warning / Critical

4.4 Finance: Research & Report Agent

Goal: Compile financial summaries from earnings calls, SEC filings, and market data for analysts.

- Tools: `search_web`, `parse_pdf`, `query_market_data`, `generate_chart`, `export_to_docx`
- Memory: Analyst preferences and past report templates stored in user profile
- Integration: Bloomberg feed, SEC EDGAR API, internal SharePoint
- Output: Auto-generated Word report with charts, ready for analyst review

5. Best Practices & Production Checklist

5.1 Safety & Guardrails

- Define hard limits in system prompt (e.g., max refund, no PII sharing).
- Implement a human-in-the-loop (HITL) step for high-stakes actions.
- Rate-limit agent API calls to prevent runaway loops.
- Use structured output schemas to prevent hallucinated tool parameters.
- Log every tool call and LLM response for auditability.

5.2 Performance Optimization

- Cache frequent tool results (e.g., product catalog, FAQs) to reduce latency.
- Use streaming responses for better perceived performance in chat interfaces.
- Run independent tool calls in parallel using `Promise.all()` or `asyncio.gather()`.
- Summarize long conversation histories before they hit the context window limit.

5.3 Evaluation Framework

Track these metrics in production to ensure agent quality:

Metric	Target	How to Measure
Task Completion Rate	> 90%	% of sessions reaching goal
Hallucination Rate	< 2%	Human review sample + factcheck
Escalation Rate	5–15%	% routed to human agent
Avg. Response Latency	< 3 seconds	P95 latency monitoring
CSAT Score	> 4.0 / 5.0	Post-session user survey

5.4 Security Considerations

- Sanitize all tool inputs to prevent prompt injection attacks.
- Never expose raw system prompts to end users.
- Apply OAuth 2.0 / API key scoping — agents should have least-privilege access.
- Store PII in encrypted, access-controlled systems; never in LLM context.
- Monitor for unusual tool call patterns (e.g., mass data export) as anomaly indicators.

6. Glossary

Term	Definition
AI Agent	Autonomous system that perceives, reasons, and acts to achieve a goal.
LLM	Large Language Model — the AI core that performs reasoning and generation.
RAG	Retrieval-Augmented Generation — grounding LLM output in retrieved documents.
Tool Use	Ability of an LLM to call external functions/APIs during reasoning.
Orchestrator	The agent or component that coordinates multi-agent workflows.
HITL	Human-in-the-Loop — human review/approval step in agentic pipelines.
Vector Store	Database that stores and retrieves content by semantic similarity.
System Prompt	Hidden instructions that define the agent's role, rules, and persona.

Stop Reason	Signal from LLM indicating whether it is done or needs to call a tool.
Prompt Injection	Attack where malicious input tricks the agent into ignoring its rules.

End of Document — AI Agents Product Integration Guide v1.0